GitHub Actions & YAML

A practical guide for Java, Maven and TestNG automation

Beginner-to-interview guide • Explains workflow files, triggers, jobs, steps, expressions, artifacts, permissions, parallel execution, debugging, and Jenkins-to-GitHub Actions mapping.

Based on your pipeline: `.github/workflows/testng-automation.yml`

Use this guide alongside your repository. Examples are illustrative; adapt Java version, suite path, runner OS, and Maven configuration to your project.

Contents

1. What is GitHub Actions?
2. What is YAML and how indentation works
3. What is a workflow? Workflow types and triggers
4. Workflow file structure
5. Your TestNG workflow explained line by line
6. Jobs, steps, runners, actions and shell commands
7. Inputs, variables, expressions and contexts
8. Artifacts, test reports and retention
9. Permissions, secrets and security
10. Parallel execution and dependencies
11. Jenkins versus GitHub Actions
12. Common failures and debugging
13. Best practices checklist
14. Interview questions and model answers
15. Quick reference cheat sheet

1. What is GitHub Actions?

GitHub Actions is GitHub's automation and CI/CD platform. It can run commands when an event occurs in a repository—for example, when code is pushed, a pull request is opened, or a developer manually starts a workflow.

CI means Continuous Integration: automatically building and testing code when changes are proposed or committed. CD means Continuous Delivery or Deployment: packaging or releasing the result after checks pass.

How the pieces fit together

Concept	Meaning	Example in your automation project
Workflow	The complete automation process defined in a YAML file.	Java TestNG Automation
Event / trigger	What starts the workflow.	Push to main; manual run
Job	A group of steps executed on one runner.	automation-tests
Runner	The machine that executes the job.	windows-latest
Step	One command or reusable action within a job.	mvn -B clean test-compile
Action	A reusable component published for a task.	actions/checkout
Artifact	A file or directory saved from the run.	Surefire XML reports

Mental model: Event → Workflow → Job → Runner → Steps → Reports / Artifacts

2. What is YAML?

YAML is a human-readable data format used to configure workflows. The file extension is usually .yml or .yaml; both are valid. GitHub Actions reads the YAML structure to understand events, jobs, steps, inputs and settings.

YAML basics

```
name: Java TestNG Automation

on:
  push:
    branches:
      - main

jobs:
  automation-tests:
    runs-on: windows-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Show Java version
        run: java -version
```

- Key-value pair: `name: Java TestNG Automation` means the key `name` has a text value.
- Indentation: spaces show nesting. The `push` event is nested under `on`.
- Lists: a hyphen introduces a list item, such as each step under `steps`.
- Strings: quotes are useful when values contain special characters or could be interpreted unexpectedly.
- Comments: lines beginning with # are comments and are not executed.

- Case: YAML keys and values are generally case-sensitive where the schema expects exact names. Keep input names consistent.

Most common YAML error: wrong indentation. Use spaces, not tabs, and align sibling keys at the same level.

Why is the extension .yml?

GitHub Actions discovers workflow files placed under `.github/workflows/`. A repository can contain multiple workflow files, each for a different pipeline or responsibility.

3. What is a workflow? Types and triggers

A workflow is an automated process described by a YAML file. A workflow can contain one or more jobs, and each job contains steps. Workflows are often classified by what triggers them or by the purpose they serve.

Common trigger types

Trigger	When it runs	Typical use
push	When commits are pushed to selected branches or tags.	Run build and tests after a commit.
pull_request	When a pull request event occurs.	Validate changes before merge.
workflow_dispatch	When a user manually starts the workflow in GitHub.	Choose QA/STAGING/DEV and a suite.
schedule	At a cron schedule (UTC).	Nightly regression or smoke tests.
workflow_call	When another workflow calls this reusable workflow.	Shared pipeline logic across repositories.
repository_dispatch	When an external event is sent to the repository.	Trigger from another system.
release	When a release event occurs.	Run release validation.
workflow_run	After another workflow completes.	Run a follow-up job or publish reports.

Trigger example

```
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:
    inputs:
      TEST_ENV:
        description: Environment for test execution
        type: choice
        options: [QA, STAGING, DEV]
        default: QA
        required: true
```

In this example, pushes and pull requests targeting main can start the workflow. The manual trigger exposes a form so a user can select an environment. The `inputs` block is for manual/reusable workflow inputs; a push does not show the manual form.

Workflow purpose categories

- CI workflow: compile, lint, unit test, integration test and report results.
- Regression workflow: run a larger suite on a schedule or before release.
- Deployment workflow: deploy an application or publish a report after required checks.

- Reusable workflow: standardize shared CI logic across projects.
- Release workflow: validate, package, tag or publish a release.

“Type of workflow” can mean trigger type or purpose category. In interviews, explain both.

4. Workflow file structure

```
name: Java TestNG Automation

on:
  push:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read

jobs:
  automation-tests:
    runs-on: windows-latest
    timeout-minutes: 60
    steps:
      - name: Checkout
        uses: actions/checkout@v4
      - name: Compile
        run: mvn -B clean test-compile
```

Field	Explanation
name	Readable workflow name shown in the Actions tab.
on	The event or events that start the workflow.
permissions	Controls the GitHub token's access for workflow operations.
jobs	Defines the jobs that make up the workflow.
automation-tests	A user-chosen job ID; use a simple, stable identifier.
runs-on	Selects the type of runner machine.
timeout-minutes	Maximum runtime for a job before it is cancelled.
steps	Ordered list of commands and actions within the job.
uses	Calls a reusable action, often from the GitHub Marketplace.
run	Executes a shell command or script on the runner.

5. Your TestNG workflow explained line by line

A. Workflow name and triggers

```
name: Java TestNG Automation

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:
    inputs:
      TEST_ENV:
        description: Environment for test execution
        type: choice
        options: [QA, STAGING, DEV]
        default: QA
        required: true
```

`name` is the display name. `push` runs on a push to main. `pull_request` validates proposed changes targeting main. `workflow_dispatch` enables a manual Run workflow button with user inputs.

B. Permissions

```
permissions:
  contents: read
```

This gives the workflow's built-in GitHub token read access to repository contents. It follows least privilege: grant only the permissions needed. If the workflow later creates releases, pushes commits, or deploys Pages, its permissions may need to change.

C. Concurrency

```
concurrency:
  group: testng-${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: false
```

Concurrency groups control overlapping runs. The group expression combines workflow and ref. With `cancel-in-progress: false`, a newer run does not automatically cancel an in-progress run in the same group. GitHub may still queue/replace pending runs within the group, so this is not a full build queue.

D. Job, runner and timeout

```
jobs:
  automation-tests:
    name: Execute TestNG Automation
    runs-on: windows-latest
    timeout-minutes: 60
```

A job is named `automation-tests`. `name` is its display label. `runs-on` chooses a GitHub-hosted Windows runner. The timeout caps the job at 60 minutes. GitHub-hosted runners are fresh virtual machines for each job.

E. Environment variables and inputs

```
env:
  MAVEN_OPTS: "-Xmx1024m"
  TEST_ENV: ${{ inputs.TEST_ENV || 'QA' }}
  TEST_SUITE: ${{ inputs.TEST_SUITE || 'testng.xml' }}
  RUN_PARALLEL: ${{ inputs.RUN_PARALLEL || 'false' }}
```

`MAVEN_OPTS` gives Maven a JVM heap setting. The other values read the manual workflow inputs and fall back to defaults. `inputs` is populated for manual dispatch and reusable workflow calls, not ordinary push or pull-request events; the fallback makes those events usable too.

Security tip: never put passwords, access tokens, or private keys in plain YAML. Store sensitive values as GitHub Actions secrets or use an approved identity mechanism.

F. Checkout and Java setup

```
- name: Checkout automation code
  uses: actions/checkout@v4

- name: Setup Java
  uses: actions/setup-java@v4
  with:
    distribution: temurin
    java-version: "21"
    cache: maven
```

`actions/checkout` downloads the repository code onto the runner. `actions/setup-java` installs/selects the requested Java distribution and version. Maven dependency caching can reduce repeated downloads.

G. Verify tools and compile

```
- name: Verify Java
  run: java -version

- name: Verify Maven
  run: mvn -version

- name: Compile automation code
  run: mvn -B clean test-compile
```

Version commands help debug the environment. `mvn -B` means batch mode, which is suitable for CI logs. `clean` removes previous build output. `test-compile` compiles main and test source code without executing the test phase.

H. Run TestNG tests and optional parallel mode

```
- name: Execute TestNG tests
  shell: pwsh
  run: |
    $args = @(
      "-B",
      "test",
      "-Denv=$env:TEST_ENV",
      "-Dsurefire.suiteXmlFiles=$env:TEST_SUITE"
    )

    if ($env:RUN_PARALLEL -eq "true") {
      $args += "-DthreadCount=4"
    }

    mvn @args
    if ($LASTEXITCODE -ne 0) {
      exit $LASTEXITCODE
    }
```

The PowerShell script builds a list of Maven arguments. It passes the environment as a Java system property named env and the suite path to Maven Surefire. If parallel mode is selected, it adds a thread-count property.

The workflow parameter alone does not guarantee parallel tests. Confirm the Maven Surefire plugin and TestNG suite are configured for parallel execution. The exact supported property names depend on the plugin version and configuration.

I. Upload reports even when tests fail

```
- name: Upload TestNG XML reports
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: surefire-reports
    path: "**/surefire-reports/*.xml"
    if-no-files-found: warn
    retention-days: 14
```

`if: always()` makes the step eligible to run even if earlier steps failed. The artifact action saves matching report files so they can be downloaded from the workflow run. Retention is 14 days for this artifact.

The second artifact step archives the entire Surefire report directory. It is useful for HTML, text, XML and other report files generated there. Uploading files is not the same as rendering test results in a GitHub summary.

J. Final execution summary

The final step uses `if: always()` and PowerShell to print a success/failure message. It is informational; it does not replace the Maven process exit code. A failed Maven test step should keep the job failed.

6. Jobs, steps, runners, actions and commands

Jobs versus steps

A job is a unit of work scheduled on a runner. Steps within a job execute in order on that runner and can share files. Different jobs run in parallel by default unless a dependency is declared, and each job normally gets its own runner.

```
jobs:
  compile:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: mvn -B clean test-compile

  test:
    needs: compile
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: mvn -B test
```

Here, `needs: compile` makes the test job wait for the compile job. Note that jobs use separate machines; files are not automatically carried between jobs. Upload/download artifacts or rebuild the files if needed.

Runner options

Runner	Good fit	Notes
ubuntu-latest	Java/Maven, Node, most cross-platform automation.	Usually a practical default for CI.
windows-latest	Windows-only tools or OS-dependent tests.	Use PowerShell or cmd syntax deliberately.
macos-latest	macOS-specific browser or app testing.	Often has different cost/availability limits.
Self-hosted	Private networks, licensed tools, special hardware.	Your team manages patching, security and capacity.

run versus uses

Syntax	Purpose	Example
run:	Executes a shell command.	run: mvn -B test
uses:	Invokes a reusable action.	uses: actions/checkout@v4
with:	Provides inputs to an action.	with: java-version: "21"
shell:	Selects the shell for a run script.	shell: pwsh

7. Inputs, variables, expressions and contexts

Inputs versus environment variables

Feature	Inputs	Environment variables
Purpose	Values supplied to a workflow or reusable workflow.	Values made available to a process.
Example	inputs.TEST_ENV	env: TEST_ENV: QA
Where used	Expressions in workflow configuration.	Shell as $env:TEST_ENV in PowerShell or $TEST_ENV in Bash.
Typical values	Environment, suite, boolean toggle.	Maven options, URLs, derived settings.

Expressions and contexts

GitHub Actions expressions use `${{ ... }}`. They let workflow fields read contextual information or calculate values.

```
${{ inputs.TEST_ENV }}
${{ github.ref }}
${{ github.event_name }}
${{ runner.os }}
${{ job.status }}
${{ needs.compile.result }}
${{ secrets.API_TOKEN }}
```

Context	What it provides
github	Repository, event, branch/ref and run metadata.
inputs	Workflow input values.
env	Workflow/job/step environment variables.
runner	Runner information, such as OS.
steps	Step outcomes and outputs for steps with IDs.
needs	Results and outputs of dependent jobs.
secrets	Configured secrets exposed to the workflow.

Conditions

```
if: always()
if: failure()
if: success()
if: cancelled()
if: steps.tests.outcome == 'failure'
```

`success()` is the usual default condition. `failure()` runs after a failure; `always()` allows cleanup/report steps to run even after failures or cancellation, though it should be used thoughtfully for operations that might hang. `cancelled()` checks cancellation state.

Shell syntax matters

Shell	Read an environment variable	Typical runner
Bash	$TEST_ENV	Ubuntu/macOS
PowerShell	$env:TEST_ENV	Windows PowerShell / pwsh
cmd	%TEST_ENV%	Windows command prompt

8. Artifacts, test reports and retention

An artifact is a file or directory saved from a workflow run. Artifacts are useful for reports, screenshots, logs, packaged builds and debugging data.

Artifact lifecycle

- The test framework creates files, for example `target/surefire-reports/TEST-*.xml`.
- The upload-artifact action stores selected files with the run.
- A developer opens Actions → workflow run → Artifacts to download them.
- The artifact is removed according to configured retention and repository/organization policies.

JUnit XML versus a rendered report

JUnit XML is a machine-readable test result format used by many Java test frameworks. Uploading XML makes it downloadable. To show test counts and failed tests inline in the Actions summary, add a test reporter action and configure it to read Surefire XML. A visual report such as Allure requires generating its HTML report and then uploading/publishing that report.

```
- name: Upload Surefire reports
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: surefire-reports
    path: "**/surefire-reports/**"
    if-no-files-found: warn
    retention-days: 14
```

Avoid accidentally uploading credentials, session cookies, customer data, or other sensitive material in logs, screenshots and artifacts.

9. Permissions, secrets and security

Least privilege

The `permissions` block controls what the workflow's `GITHUB_TOKEN` can do. Start with minimal permissions and add only those required by a task. For example, deploying to GitHub Pages normally needs pages: write and id-token: write on the deployment job.

Secrets

```
- name: Run tests against a protected endpoint
  env:
    API_TOKEN: ${{ secrets.API_TOKEN }}
  run: mvn -B test
```

Create secrets under repository Settings → Secrets and variables → Actions. Never hard-code a token in YAML, commit it to the repository, or print it in logs. Be cautious with pull requests from forks because secrets are normally not made available to untrusted fork code.

Third-party actions

- Use reputable actions and review their documentation and maintenance status.
- Pin actions to a full commit SHA when your organization's security policy requires stronger supply-chain protection.
- Do not run untrusted pull-request code on a self-hosted runner with valuable credentials.
- Use protected environments and required reviewers for production deployments.

10. Parallel execution and dependencies

Parallel jobs

```
jobs:
  api-tests:
    runs-on: ubuntu-latest
    steps:
      - run: mvn -B test -Dgroups=api

  ui-tests:
    runs-on: ubuntu-latest
    steps:
      - run: mvn -B test -Dgroups=ui
```

These jobs can run concurrently because neither depends on the other. Each needs its own checkout and setup steps.

Sequential jobs with needs

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - run: mvn -B clean package

  test:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - run: echo "Run tests after build"
```

Use needs to define dependencies. If a required job fails, downstream jobs are skipped by default unless an appropriate condition is specified.

Matrix strategy

```
strategy:
  matrix:
    java: ["17", "21"]
    os: [ubuntu-latest, windows-latest]

runs-on: ${{ matrix.os }}
```

A matrix creates multiple job combinations—in this example, Java 17 and 21 on Ubuntu and Windows. That creates four combinations unless exclusions or fail-fast behavior change the plan.

Parallel test execution inside TestNG

GitHub Actions parallel jobs are different from TestNG threads. A workflow matrix or multiple jobs parallelizes work across runners. TestNG's parallel mode runs tests or methods concurrently inside one test process, subject to the TestNG XML and Maven Surefire configuration. Both can be used, but avoid oversubscribing resources or sharing mutable test data.

11. Jenkins versus GitHub Actions

Jenkins	GitHub Actions	Key difference
Jenkinsfile	.github/workflows/*.yml	Pipeline definition format.
agent any	runs-on	Selects an execution machine.
stage	Often represented by a job or named steps	Not a one-to-one mapping in every design.
steps { bat '...' }	run: ...	Shell syntax depends on runner OS.
parameters	workflow_dispatch inputs	Manual inputs are configured in workflow YAML.
environment {}	env:	Both set variables, with different scope/context rules.
post { always }	if: always()	Conditional step/job execution.
archiveArtifacts	upload-artifact action	Stores downloadable run files.
junit()	Reporter action or uploaded XML	Upload alone does not render a test summary.
buildDiscarder	Repository/organization run and artifact retention settings	Retention settings are configured separately.

Interview answer: Jenkins is a general automation server with a large plugin ecosystem and commonly self-managed agents. GitHub Actions is integrated into GitHub repositories and uses hosted or self-hosted runners, YAML workflows, events and reusable actions.

12. Common failures and debugging

Symptom	Likely cause	What to check
Workflow does not start	Wrong file location, invalid YAML, branch mismatch or event config.	Ensure file is under .github/workflows and inspect Actions tab.
Maven not found	Java/Maven setup or PATH issue.	Check setup-java, mvn -version and runner OS.
Wrong suite runs	Suite path/property is incorrect.	Check suite XML path and Surefire plugin version/config.
Parallel flag has no effect	Surefire/TestNG parallel settings missing.	Check plugin configuration and testng.xml.
No reports uploaded	Tests did not generate files or path glob is wrong.	Inspect target/surefire-reports and artifact step logs.
YAML parse error	Indentation, colon or quoting problem.	Use spaces; quote values containing special characters.
PowerShell command fails	Bash syntax used on Windows or exit code mishandled.	Use $env:VAR, correct shell and preserve $LASTEXITCODE.
Workflow token denied	Missing permissions.	Review permissions and action requirements.

Debugging checklist

- Open the failed run in Actions and expand the first failed step.
- Verify the event, branch, runner OS, Java version and Maven version.
- Check that required files exist at the expected relative paths.
- Inspect Maven exit code and Surefire XML output.
- Use artifacts to download reports and screenshots.
- Keep logs useful, but never print secrets.

13. Best-practices checklist

☐ Keep workflow files in .github/workflows/ and give them clear names.

☐ Use explicit Java versions instead of relying on runner defaults.

☐ Use actions/setup-java with Maven caching for Java projects.

☐ Set sensible timeouts to avoid hanging jobs.

☐ Use least-privilege token permissions.

☐ Upload test reports with if: always() so failures are diagnosable.

☐ Separate test execution failure from report collection.

☐ Use workflow_dispatch for controlled manual environment selection.

☐ Use protected GitHub environments for sensitive deployments.

☐ Do not assume a parameter enables parallel execution unless the test runner is configured.

☐ Prefer stable suite paths and validate user-provided values.

☐ Review artifact contents and retention for security and storage.

14. Interview questions and model answers

Q1. What is GitHub Actions?

GitHub Actions is a CI/CD automation platform integrated with GitHub. Workflows are defined in YAML and run in response to repository events or manual/scheduled triggers.

Q2. What is a workflow?

A workflow is the complete automated process stored in a YAML file under .github/workflows. It defines triggers, jobs, permissions and other settings.

Q3. What is the difference between a job and a step?

A job is a set of steps executed on a runner. Steps run in order within the job. Different jobs run on separate runners and may execute in parallel unless dependencies are configured.

Q4. What is a runner?

A runner is the machine that executes workflow jobs. It can be GitHub-hosted, such as ubuntu-latest or windows-latest, or self-hosted and managed by the organization.

Q5. How do you run a workflow manually with parameters?

Use workflow_dispatch and define inputs such as a choice for environment, a string for suite path, and a boolean for parallel mode. The user selects values from the Actions UI.

Q6. How do you run cleanup or upload reports when tests fail?

Use if: always() on the reporting or cleanup step. Keep the test command's failure status so the workflow still accurately reports that tests failed.

Q7. Does upload-artifact display test results in the GitHub UI?

It stores files for download. To render test counts and failures in the run summary, configure a compatible test reporter; for Allure, generate the HTML report and publish or upload it.

Q8. How do you configure parallel execution?

At workflow level, use independent jobs or a matrix. Within TestNG, configure parallel execution and thread counts in TestNG/Surefire. These are different layers and must be configured independently.

Q9. How do you pass secrets safely?

Store credentials as GitHub Actions secrets and reference them through the secrets context, ideally via an environment variable. Never commit or print credentials.

Q10. What is the purpose of permissions?

The permissions block limits the GITHUB_TOKEN capabilities. Least privilege reduces the impact if a workflow or action is compromised.

Q11. How do you retain reports?

Use actions/upload-artifact with a name, path, if-no-files-found and retention-days. Repository or organization policies may constrain retention.

Q12. How does GitHub Actions differ from Jenkins?

Jenkins commonly uses a Jenkinsfile, stages, agents and plugins, often on self-managed infrastructure. GitHub Actions uses YAML workflow files, events, jobs, runners and reusable actions integrated with GitHub.

Q13. Why use setup-java with cache: maven?

It selects a predictable JDK version and caches Maven dependencies to reduce repeated downloads and speed up builds.

Q14. What is the purpose of needs?

needs defines job dependencies. A downstream job waits for its required jobs to finish and normally does not run if a dependency fails or is skipped.

Q15. Why can a workflow pass compilation but fail during tests?

Compilation only proves code compiled. Runtime configuration, environment variables, external services, test data, browser setup or assertions can still fail during execution.

15. Quick reference cheat sheet

Need	Syntax
Manual run	on: workflow_dispatch:
Push trigger	on: push:
Choose runner	runs-on: ubuntu-latest
Set timeout	timeout-minutes: 60
Run command	run: mvn -B test
Use action	uses: actions/checkout@v4
Pass action input	with: java-version: "21"
Set environment variable	env: TEST_ENV: QA
Read manual input	${{ inputs.TEST_ENV }}
Read secret	${{ secrets.API_TOKEN }}
Always run report step	if: always()
Run after failure	if: failure()
Job dependency	needs: build
Upload artifact	uses: actions/upload-artifact@v4
Retention	retention-days: 14

Final practical notes

- This guide explains the workflow structure and the TestNG/Maven example. Verify the Java version, Surefire property names, suite XML location and TestNG parallel configuration against your actual pom.xml.
- GitHub-hosted runner images and action versions evolve. Before using a workflow in production, check the official documentation for the current action versions and runner support.
- A workflow's YAML syntax and the test framework's configuration are separate concerns: valid YAML does not guarantee Maven or TestNG will interpret every property as intended.

Official references to keep handy

• GitHub Actions documentation: https://docs.github.com/actions

• Workflow syntax: https://docs.github.com/actions/writing-workflows/workflow-syntax-for-github-actions

• Workflow events: https://docs.github.com/actions/writing-workflows/choosing-when-your-workflow-runs/events-that-trigger-workflows

• Artifacts: https://docs.github.com/actions/using-workflows/storing-workflow-data-as-artifacts

• setup-java action: https://github.com/actions/setup-java

• Maven Surefire Plugin: https://maven.apache.org/surefire/maven-surefire-plugin/